

UNITED STATES PATENT APPLICATION

Title: SYSTEM AND METHOD FOR ATOMIC AUTHORIZATION AND IMMUTABLE STATE ENFORCEMENT IN NON-DETERMINISTIC AUTONOMOUS AGENT ENVIRONMENTS

Inventors: [Your Name], [Team Members]

Assignee: [Company Name]

Date: November 22, 2025

CROSS-REFERENCE TO RELATED APPLICATIONS

[0001] This application claims the benefit of U.S. Provisional Application No. 63/XXX,XXX, filed on [Date], the entire contents of which are incorporated herein by reference.

BACKGROUND OF THE INVENTION

1. Field of the Invention

[0002] The present invention relates generally to distributed computing security and autonomous systems. More particularly, it relates to a governance kernel that enforces atomic authorization, resource quotas, and immutable logging for non-deterministic autonomous software agents, thereby preventing race conditions and ensuring verifiable accountability in untrusted execution environments.

2. Description of Related Art

[0003] The rapid proliferation of autonomous AI agents—software entities capable of reasoning, planning, and executing multi-step tasks without human intervention—has introduced significant security and governance challenges. Unlike traditional deterministic microservices, autonomous agents exhibit non-deterministic behaviors, including recursive self-prompting, tool use loops, and unpredictable resource consumption.

[0004] Existing Identity and Access Management (IAM) systems, such as OAuth2 or OPA (Open Policy Agent), operate on a “check-then-act” paradigm. An authorization service evaluates a request and returns a token or boolean approval. The agent then proceeds to execute the action. In high-concurrency autonomous environments, this separation creates a critical race condition: an agent may be authorized to consume a resource (e.g., API budget, compute quota) that was depleted by another agent in the milliseconds between the check and the execution.

[0005] Furthermore, current audit logging mechanisms are typically implemented as “side effects”—asynchronous processes that record actions after they occur. If the logging service fails or is bypassed, the action may still proceed, breaking the chain of custody. This lack of “technical guarantees” for accountability renders autonomous agents unsuitable for high-stakes environments (e.g., financial trading, critical infrastructure).

[0006] Prior art systems like IBM Governance Agents or Boomi Agent Control Tower rely on static policy rules or post-hoc monitoring. Similarly, ACID (Atomicity, Consistency, Isolation, Durability) transaction systems and two-phase commit (2PC) protocols utilize write-ahead logs to ensure database consistency, while blockchain technologies use append-only ledgers for audit trails and non-repudiation.

[0006a] However, these existing technologies fail to address the specific challenges of governing non-deterministic autonomous agents at the network boundary. Standard ACID transactions assume a known, bounded transaction scope and do not intercept external API calls from a non-deterministic agent. Blockchain ledgers, while providing immutability, typically suffer from latency (e.g., block confirmation times) that effectively precludes their use for real-time, synchronous authorization of high-frequency agent tool calls.

[0006b] A person of ordinary skill in the art of Identity and Access Management (IAM) or distributed logging would not have been motivated, absent the specific autonomous agent governance problem, to combine these technologies into a single, fail-closed atomic transaction at the network edge. Traditional IAM is designed for high availability and low latency, operating on a stateless “check-then-act” basis. Traditional logging sidecars are explicitly designed to be “fail-open” (i.e., if the logger fails, the application continues) to prevent observability tools from impacting production availability. Extending transactionality to enforce a “hard” fail-closed requirement—where a logging failure mechanically blocks the primary workload—introduces performance penalties (increased tail latency waiting for log commits) and availability risks that are contrary to conventional system design principles. This counter-intuitive architecture is, however, strictly necessary to enforce verifiable accountability for autonomous agents with unbounded, non-deterministic behaviors.

SUMMARY OF THE INVENTION

[0007] The present invention overcomes the deficiencies of the prior art by providing a **Governance Kernel** that implements an **Atomic Governance Transaction (AGT)**. The system mechanically couples cryptographic identity verification, state-aware policy evaluation (checking and locking resources), and immutable ledger commitment (write-ahead logging) into a single, indivisible atomic unit.

[0008] In one embodiment, a computer-implemented method for authorizing actions of an autonomous agent comprises: intercepting an action request from the agent via a sidecar proxy; locking a resource state associated with the agent in a high-performance state engine; evaluating a policy against the locked resource state; generating an immutable log entry comprising a cryptographic hash

of a previous log entry; writing the log entry to a tamper-evident storage medium; and **only upon receiving a write confirmation**, updating the resource state and permitting the action request.

[0009] This approach eliminates the “check-then-act” race condition by ensuring that resource deduction happens in the same atomic transaction as authorization. It further guarantees accountability by making the immutable log entry a prerequisite for execution, rather than a side effect.

[0010] Technical improvements include:

- **Prevention of Inconsistent Distributed State:** Through pessimistic locking of a locked quota counter maintained in shared memory, the system prevents “double-spend” of agent budgets. For example, in a scenario with $N=100$ concurrent agents competing for the last unit of a shared quota ($\text{quota}=1$), traditional check-then-act architecture exhibits a race window during which multiple agents may receive approval, resulting in quota overspend. The AGT system guarantees exactly one approval through atomic state lock and commit operations regardless of concurrency level.
- **Guaranteed Auditability:** Enforcement of write-ahead logging as a technical prerequisite for state transitions using a governance log record with fields (`request_id`, `policy_hash`, `state_delta`, `timestamp`, `agent_identity`, `prev_record_hash`). The state engine commit operation is blocked until successful acknowledgment from tamper-evident storage, eliminating the possibility of state changes without corresponding audit records.
- **Reduced Latency:** Reduction of policy resolution round-trip latency by consolidating previously distributed service calls (identity verification, quota check, policy evaluation, and log write) into a single atomic operation with co-located state management in the governance kernel, reducing network round-trips from 4+ to 1.
- **Fail-Closed System Behavior:** Guaranteed denial of agent actions upon any governance component failure through a network interceptor that blocks transmission until log commit succeeds, contrasting with conventional fail-open monitoring sidecars.

BRIEF DESCRIPTION OF THE DRAWINGS

[0011] **FIG. 1** is a block diagram illustrating the high-level architecture of the Governance Kernel system. [0012] **FIG. 2** is a sequence diagram showing the Atomic Governance Transaction (AGT) flow. [0013] **FIG. 3** illustrates the data structure of the cryptographically chained decision ledger. [0014] **FIG. 4** is a flowchart of the state-aware policy evaluation logic (“Check-and-Lock”). [0015] **FIG. 5** illustrates the interceptor mechanism within a Kubernetes sidecar pattern. [0016] **FIG. 6** shows the rollback mechanism in the event of a ledger write failure. [0017] **FIG. 7** is a timing diagram contrasting the race condition window in a baseline architecture versus the atomic enforcement in the present invention. [0018] **FIG. 8** illustrates a missing log scenario in a baseline architecture versus the fail-closed guarantee of the present invention. [0019] **FIG. 9** illustrates a preferred embodiment of the system deployed as a sidecar container within a Kubernetes pod.

DETAILED DESCRIPTION OF THE INVENTION

[0017] The following detailed description refers to the accompanying drawings. The same reference numbers in different drawings identify the same or similar elements.

4.1 Baseline Architecture and Deficiencies

[0017a] **FIG. 7** and **FIG. 8** illustrate a baseline “check-then-act” architecture common in the prior art. In such systems, a separate Identity Provider (IdP) and Quota Service exist. An agent first requests authorization (Check). If approved, the agent proceeds to execution (Act). [0017b] As shown in **FIG. 7**, a race condition occurs when Agent A and Agent B concurrently check a quota of 1. Both receive approval at time t_0 , and both execute at time t_1 , resulting in a resource overspend ($usage = 2$). [0017c] As shown in **FIG. 8**, logging is typically an asynchronous “fire-and-forget” sidecar. If the logging sidecar crashes or the network partition occurs, the agent’s action still proceeds, resulting in a state change with no audit record (“Missing Log”).

4.2 Key Definitions

[0017d] **“Autonomous Agent”**: A software entity capable of non-deterministic behavior, including but not limited to Large Language Model (LLM) based systems exhibiting hallucination, recursive self-prompting loops, and unpredictable resource consumption patterns. This definition includes, in some embodiments, agents with partially or fully deterministic behavior. [0017e] **“Atomic Governance Transaction (AGT)”**: A specific architectural pattern comprising the sequential and mechanically coupled steps of: (1) cryptographic identity verification; (2) pessimistic state locking; (3) policy evaluation against locked state; (4) tentative state update calculation; (5) immutable log entry generation; (6) write-ahead log commitment with acknowledgment wait; and (7) commit or rollback based on log write success. [0017f] **“Tamper-Evident Storage”**: A storage medium configured as write-once-read-many (WORM) with cryptographic integrity verification. Non-limiting examples include WORM-compliant object storage (e.g., AWS S3 Object Lock), blockchain-based append-only logs, Merkle tree data stores, or equivalent systems providing immutability and cryptographic verification.

4.3 System Architecture

[0018] Referring to **FIG. 1**, the system comprises three tightly coupled components: the **Interceptor** (102), the **State Engine** (104), and the **Decision Ledger** (106). The Interceptor (102) is a proxy (e.g., gRPC/Envoy extension) that intercepts all outbound signals from an Agent (100). It halts execution until it receives a cryptographically signed AGT Token from the State Engine.

[0018] Referring to **FIG. 1**, the system comprises three tightly coupled components: the **Interceptor** (102), the **State Engine** (104), and the **Decision Ledger** (106). The Interceptor (102) is a proxy (e.g.,

gRPC/Envoy extension) that intercepts all outbound signals from an Agent (100). It halts execution until it receives a cryptographically signed AGT Token from the State Engine.

[0019] The State Engine (104) is a high-performance, in-memory state machine (e.g., implemented in Rust or Go) that holds the “Global State” of all agent quotas, budgets, and permissions. Unlike stateless policy engines, the State Engine (104) maintains real-time counters and locks.

[0020] The Decision Ledger (106) is an append-only, cryptographically chained data store (e.g., S3 Object Lock with hash chaining). It serves as the “write-ahead log” for the system.

4.4 The Atomic Governance Transaction (AGT)

[0021] **FIG. 2** illustrates the AGT flow. When Agent (100) attempts an action (e.g., `API_Call(X)`), the Interceptor (102) halts the call and sends a `Query(AgentID, ActionX, CostY)` to the State Engine (104).

[0022] Upon receipt, the State Engine (104) performs a **Cryptographic Verification** (Step 202) of the Agent’s signature. If valid, it proceeds to the **State-Aware Evaluation** (Step 204).

[0023] As shown in **FIG. 4**, the State-Aware Evaluation involves locking the `State(AgentID)` (Step 402). The logic evaluates: `IF (Policy allows ActionX) AND (Budget >= CostY)`. If true, it tentatively calculates `NewBudget = Budget - CostY` but does *not* yet commit it.

[0024] The State Engine (104) then generates a **Log Entry** (Step 206). As shown in **FIG. 3**, the Log Entry (300) includes a `prev_hash` (302) linking it to the previous entry, ensuring a tamper-evident chain.

[0025] The Log Entry is written to the Decision Ledger (106) (Step 208). The State Engine (104) **waits** for a confirmation (ACK) from the Ledger. This is the critical “Write-Ahead” step.

[0026] If the ACK is received (Step 210), the State Engine (104) commits the `NewBudget` to memory, unlocks the state, and returns an `APPROVE` signal to the Interceptor (102). If the ACK fails (timeout or error), the State Engine (104) rolls back the `NewBudget`, unlocks the state, and returns `DENY`.

4.5 Enablement Implementation

4.5.1 State Engine Implementation

[0026a] The State Engine (104) implements pessimistic locking for resource counters. Locking granularity may be configured per-agent, per-budget, or per-tenant. To prevent deadlocks and ensure system responsiveness, the locking mechanism employs configurable timeouts. [0026b] Critical data structures include a **locked quota counter** maintained in shared memory (e.g., utilizing atomic Compare-And-Swap (CAS) operations or distinct mutex locks). This ensures that for `N` concurrent agents requesting access to a resource with a remaining quota of 1, the system guarantees exactly one successful acquisition, preventing the race conditions illustrated in **FIG. 7**.

4.5.2 Decision Ledger Write Path

[0026c] The Decision Ledger write path ensures idempotency and resilience. The log entry generation logic assigns a unique transaction ID (request_id) to handling duplicate requests. [0026d] A specific **governance log record** schema comprises: request_id, policy_hash (optimizing validation of policy versioning), state_delta (capturing the resource change), timestamp (for temporal ordering), agent_identity, and prev_record_hash. [0026e] The write operation includes retry logic with exponential backoff for transient failures. However, if the retry limit is exhausted, the transaction strictly behaves as **fail-closed**, triggering the rollback described in Step 212.

4.5.3 Interceptor Integration and Fail-Closed Pattern

[0026f] The Interceptor (102) is deployed as a network proxy. Unlike conventional logging or monitoring sidecars which are designed as “fail-open” (i.e., sidecar failure does not impact the application), the present governance sidecar is explicitly “fail-closed.” [0026g] **Operational Integration:** The Interceptor blocks any outbound traffic that does not possess a valid AGT Token. In container orchestration platforms (e.g., Kubernetes, Docker Swarm, or compatible systems), the Interceptor may be integrated with liveness and readiness probes. Repeated failures to complete an AGT (indicating an inability to write to the ledger) cause the Interceptor to report an unhealthy status, triggering the orchestration platform to restart or quarantine the agent pod. This converts governance failures into standard operational health events.

4.5.4 Multi-Agent Consensus with Peer Review

[0026h] In a preferred embodiment, the system supports a “Negotiated Consensus” mode for high-risk actions. An Operation Agent submits a proposed plan to a designated Governance Agent (Peer Reviewer) prior to execution. The Governance Agent evaluates the plan against safety hyperparameters. [0026i] This process may involve multiple cycles of negotiation: if the Governance Agent rejects the proposal, it returns feedback; the Operation Agent refines the plan and resubmits. The Governance Kernel is configured to reject any AGT request that does not include the cryptographic signature of the Governance Agent, thereby enforcing the Governance Agent’s final authority over the pattern of execution.

Technical Advantages

[0027] This architecture transforms governance from a “soft” compliance process into a “hard” technical constraint. The **mechanical coupling** of state, policy, and logging ensures that the system fails closed (securely) in the event of any component failure, a critical requirement for autonomous systems operating in untrusted environments.

CLAIMS

What is claimed is:

1. A computer-implemented method for authorizing actions of an autonomous agent, comprising: intercepting, via a proxy, an action

request from the autonomous agent; locking, by a state engine, a resource state associated with the autonomous agent; evaluating, by the state engine, a policy against the locked resource state to determine a tentative authorization result; generating, by the state engine, an immutable log entry comprising a cryptographic hash of a previous log entry and the tentative authorization result; writing the immutable log entry to a tamper-evident storage medium; and **only upon receiving a confirmation of the writing step:** updating the resource state in the state engine; and permitting the action request to proceed.

2. The method of claim 1, wherein the locking, evaluating, writing, and updating steps are performed as a single atomic transaction, such that a failure in the writing step causes a rollback of the tentative authorization result and a denial of the action request.
3. The method of claim 1, wherein the resource state comprises a real-time budget counter, and the updating step comprises decrementing the budget counter by a cost associated with the action request.
4. The method of claim 1, wherein the immutable log entry further comprises a digital signature of the autonomous agent, providing non-repudiation of the action request.
5. The method of claim 1, wherein the proxy is a sidecar process running in the same execution environment as the autonomous agent, configured to intercept all outbound network traffic.
6. A system for enforcing atomic governance in an autonomous agent environment, comprising: an interceptor configured to intercept action requests from an autonomous agent; a decision ledger configured to store immutable log entries; and a governance kernel configured to: receive an intercepted action request; lock a resource state associated with the agent; validate the request against a policy and the resource state; write a log entry to the decision ledger; and permit the action request only after receiving a write confirmation from the decision ledger.
7. The system of claim 6, wherein the decision ledger utilizes a cryptographic hash chain to link consecutive log entries, thereby rendering the log tamper-evident.
8. The system of claim 6, wherein the governance kernel is further configured to enforce a timeout on the write confirmation, wherein expiration of the timeout triggers a denial of the action request.
9. A non-transitory computer-readable medium storing instructions that, when executed by a processor, cause the processor to perform operations comprising: receiving a request from an autonomous agent to perform an operation; verifying a cryptographic identity of the autonomous agent; accessing a real-time state store to verify sufficient resource quota for the operation; generating a signed audit record of the request and the verification result; transmitting the signed audit record to an immutable storage service; waiting for a success acknowledgement from the immutable storage service; and executing the operation only if the success acknowledgement is received.

10. The non-transitory computer-readable medium of claim 9, wherein the operations further comprise rolling back any tentative state changes if the success acknowledgement is not received.
11. The computer-implemented method of claim 1, wherein the proxy is deployed as a sidecar process in a container orchestration platform (e.g., Kubernetes, Docker Swarm, or compatible system), and wherein any error in the state engine or decision ledger writing step causes immediate denial of the action request and prevents execution, and further wherein repeated governance failures trigger a health check failure in the orchestration platform.
12. A system for enforcing atomic governance in autonomous agent environments deployed on a container orchestration platform, comprising: a sidecar proxy deployed in the same execution context as an autonomous agent; a tamper-evident storage medium (e.g., WORM-compliant object storage, blockchain-based append-only logs); a cryptographic identity provider; and a governance kernel configured to atomically authorize action requests by validating identity, locking a resource quota, and persisting an audit record to the tamper-evident storage before permitting the action request to proceed.
13. A method for governing autonomous agents in regulated environments including financial transaction processing, critical infrastructure control, or compliance-sensitive workloads, wherein the fail-closed atomic governance transaction satisfies regulatory requirements for audit completeness under frameworks including but not limited to SOC2, PCI-DSS, SOX, GDPR, or equivalent standards.
14. The method of claim 1, further comprising a peer review negotiation step prior to the locking step, wherein the autonomous agent iterates on a proposed action plan with a second, governance-focused autonomous agent until a consensus is reached, and wherein the state engine requires a cryptographic signature from said governance-focused autonomous agent as a mandatory condition for processing the log entry and updating the resource state.

ABSTRACT OF THE DISCLOSURE

A system and method for atomic authorization and immutable state enforcement in autonomous agent environments. A governance kernel intercepts action requests from non-deterministic agents and performs a “check-and-lock” operation on the agent’s resource state. The kernel generates a cryptographically chained log entry and writes it to an immutable ledger. The action is permitted and the state updated only upon confirmation that the log entry has been successfully persisted. A multi-agent consensus mechanism requires cryptographic countersignatures from peer reviewer agents following iterative negotiation cycles. This atomic transaction model eliminates race conditions between authorization and resource consumption and guarantees that no action occurs without a verifiable audit trail, thereby enabling secure deployment of autonomous agents in high-stakes environments.